

Course title: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 15 tasks

Page 1 of 12 pages

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one file per exercise**, with **these exact names**:
 - For exercises 1 to 3, `exZZ-library.cpp`, where `ZZ` ranges from 01 to 03 (i.e., one per exercise);
 - For exercise 4, `ex04-library.zip`, which must contain `ex04-library.cpp` and `ex04-library.h`.
3. You can test your solutions by uploading them on Autolab, under “Exam December 2023” at:
<https://autolab.compute.dtu.dk/courses/02393-E23/assessments>
4. You can test your solutions on Autolab as many times as you like. *Uploads on Autolab are not official submissions* and will not affect your grade.
5. In case Autolab is malfunctioning, you can test your solution by comparing your output with `exZZ-main-output.txt`, which is enclosed in the ZIP archive for each handout.
6. Additional tests may be run on your submissions after the exam.
7. Feel free to add comments to your code.
8. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. BATTLESHIP

Alice is implementing a variant of the old game *Battleship* (also known as Sea Battle): the first player places up to 9 ships in a sea with a map made of tiles. Each ship can occupy up to 4 consecutive tiles, and it can be placed either horizontally or vertically. Each ship must be numbered differently from the other ships. Ships must not overlap. Then, the second player has to sink all the ships by launching missiles targeting a specific tile. A map of size 8×16 might look like **Fig. 1** on the right:

- ' ' is an empty tile;
- '3' is a tile occupied by ship number 3, which has not been hit by a missile;
- 'X' is a tile occupied by a ship, which has been hit by a missile.

After the first player has placed all the ships, they become hidden. The sea with hidden ships is drawn as shown in **Fig. 2** on the right:

- tiles that are not hit by a missile are shown as '?';
- if a tile was hit by a missile, then:
 - if the tile is occupied by a ship, it is shown as 'X';
 - otherwise, it is shown as ' '.

Alice has written some code: a test program is in the file `ex01-main.cpp`, and an (incomplete) implementation is in `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A tile of the sea is represented as a `struct Tile` with two fields named `ship` and `hit`, which represent, respectively:

- the number associated to the ship occupying that tile; 0 indicates that no ship occupies that tile;
- whether the tile has been hit by a missile.

Alice's code already includes a function (that you must not modify):

- `void deleteSea(Tile **sea, unsigned int m)`

This function deallocates the sea.

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

This exercise continues on the next page...

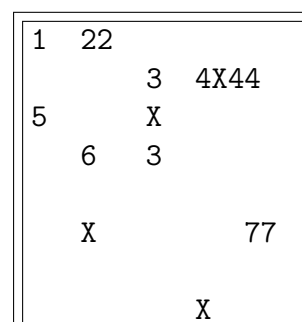


Fig. 1. 8×16 sea with all tiles visible.

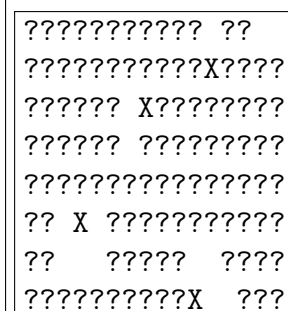


Fig. 2. The sea above with hidden ships, some of which have been hit.

(a) Implement the function:

```
Tile** createSea(unsigned int m)
```

It must return an array of $m \times 2m$ Tiles, i.e., `Tile**`. It must allocate the required memory, and initialise each tile with `ship` set to 0 and `hit` set to `false`.

(b) Implement the function:

```
void displaySea(Tile **sea, unsigned int m,
                bool reveal)
```

The function must print on screen the contents of all the tiles of the sea `sea` of size $m \times 2m$, according to the explanation at the beginning of this exercise: when `reveal` is set to `true`, the sea must be displayed as shown in **Fig. 1** of page 2; when `reveal` is set to `false`, the sea must be displayed as shown in **Fig. 2** of page 2.

(c) Implement the function:

```
bool launchBigMissile(Tile **sea, int m, int r, int c)
```

where `sea` is a sea of size $m \times 2m$, and `r` and `c` are a row and column position of the tile targeted by the missile.

The function tries to launch a missile targeting the tile at position (r, c) . The function must check whether:

- The position (r, c) is within the boundaries of the sea;
- the tile has not been hit.

If any of these conditions is *not* satisfied, the function must return `false` without altering the sea. If all conditions are satisfied, the function must:

- update the sea `sea` by marking as hit:
 - the tile at position (r, c) ;
 - the tiles that are at most 1 tile far - horizontally or vertically - from position (r, c) , if they exist.
- return `true`.

This exercise continues on the next page...

Example. Assume that `sea` is the sea shown on page 2, **Fig. 2**: it has size 8×16 .

- `launchBigMissile(sea, 8, 2, 6)` must return `false` (tile at position (2,6) has already been hit);
- `launchBigMissile(sea, 8, 4, 15)` must return `true` (after updating the sea by marking the tiles at position (4,15), (3,15), (5,14), (4,14) as hit);
- `launchBigMissile(sea, 8, 4, 14)` must return `false` (tile at position (4,14) has been hit when `launchBigMissile(sea, 8, 4, 15)` was invoked);
- `launchBigMissile(sea, 8, 3, 14)` must return `true` (after updating the sea by marking the tiles at position (3,14), (2,14), (3,13) as hit);

(d) Implement the function:

```
bool isGameOver(Tile **sea, int m)
```

where `sea` is a sea of size $m \times 2m$.

The function must check if all tiles occupied by a ship have been hit. If so, the function must return `false`. Otherwise, the function must return `true`.

EXERCISE 2. PLAYLIST

Bob is writing a program to manage the songs aired by a radio station. In particular, songs queue in a playlist, where they are removed once they are played. Normally, the playlist works in FIFO (first-in-first-out) order: the oldest song being added to the playlist will be the first one to be played. However, one can decide to reorder the playlist by shifting a listed song before or after the others. Bob decides to represent the playlist as a linked list, where the head of the list represents the song that will be played first, and the tail the song that will be played last.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A playlist element is represented as a `struct Song` with 4 fields: `title`, `artist`, `genre`, `duration`, and `next`. Such fields represent, respectively, the title of the song, the artist, the genre and the duration in seconds, and a pointer to the next song in the playlist (which will be played after the current one) (or `nullptr` when there are no more songs). An empty playlist is represented as a `Song*` pointer equal to `nullptr`. Bob's code already includes a function to print the content of the playlist on screen:

```
void displayPlaylist(Song *s)
```

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
unsigned int totalDuration(Song *s, string genre)
```

which computes the total duration of all songs of the specified `genre` in the playlist `s` and returns it.

This exercise continues on the next page...

(b) Implement the function:

```
Song* find(Song *s, unsigned int duration)
```

which returns a new playlist containing all Songs in the playlist `s` that are longer than the specified `duration`, preserving the order in which they appear in `s`. **Important:** the function must return a new playlist, where each `Song` is a dynamically-allocated copy of its original from `s`; the function must *not* modify `s`.

(c) Implement the function:

```
Song* split(Song *&s, unsigned int pos)
```

which tries to modify the playlist `s` by moving the song located from position `pos` onwards into a new playlist, which will be returned by that function. Note that the position of songs start from 1.

The function must check if a song exists at position `pos`. If this condition is *not* satisfied, the function must return an empty playlist without altering `s`. If this condition is satisfied, the function must update `s` by removing all the songs from position `pos` onwards. In addition, it must add these songs to a new playlist, which will be returned by the function. Also, the order of the songs in the new playlist must be the same as the one in the original playlist.

Example. Assume that `s` contains the following items (from the head to the tail):

- Billie Jean, Micheal Jackson, Pop, 232
- Sandstorm, Darude, Dance, 225
- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196

Then, `split(*s, 5)` must return `false` since there is no song at position 5. Conversely, `split(*s, 3)` must change `s` to reflect the following order:

- Billie Jean, Micheal Jackson, Pop, 232
- Sandstorm, Darude, Dance, 225

In addition, it must return a new playlist containing the following items:

- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196

EXERCISE 3. PARKING AREA

Claire works in a municipality, which has recently installed several cameras to monitor a public parking area. She is writing a class `ParkingArea` to manage the information about the parking spaces in that car park, and the vehicles parked in there. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Claire has represented the information about a vehicle occupying a parking space using a `struct Vehicle`, with 2 fields:

- `plate`: the license plate of the vehicle;
- `owner`: the name of the owner of the vehicle.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 9*.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `enum Category { CAR, MOTORBIKE, TRUCK }` — an enumeration to specify the category of a vehicle.
- `map<string,Category> parkingSpaces` — a mapping from `strings` (parking space IDs) to `Category` (info about the category of vehicle that should occupy the parking space);
- `map<string,Vehicle> parkingOccupancy` — a mapping from `strings` (parking space IDs) to instances of `Vehicle` (info about the vehicle occupying the parking space, if any). When a parking space ID does not appear in this mapping, it means that the parking space is empty.

Claire has already implemented the default constructor of `ParkingArea`, which creates an internal database with all the parking spaces installed. She has also implemented the methods `display()`, which shows whether a parking space is occupied, and what vehicle occupies it, and `categoryToString()`, which converts a `Category` into a `string`.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

This exercise continues on the next page...

- (a) Implement the following method to register a vehicle occupying a parking space:

```
void ParkingArea::park(string parkingSpaceID, string plate,
    string owner, Category category)
```

The method must work as follows:

- (a) if `parkingSpaceID` is *not* in `parkingSpaces`, do nothing;
- (b) otherwise:
 - if the parking space `parkingSpaceID` is for vehicles belonging to a different `category`, do nothing;
 - otherwise:
 - if the parking space `parkingSpaceID` is already occupied by a vehicle, do nothing;
 - if the parking space `parkingSpaceID` is available, then update the map `parkingOccupancy` to register the vehicle occupying that parking space.

- (b) Implement the following method to register vehicles leaving a parking space:

```
void ParkingArea::leave(vector<string> plates)
```

The method updates the map `parkingOccupancy` to remove information on the vehicles whose license plate is contained in the given collection `plates`.

- (c) Implement the following method to count empty parking spaces:

```
unsigned int ParkingArea::getEmptySpaces(Category category)
```

The method returns how many empty parking spaces are available for a vehicle of the specified `category`.

For example, suppose that there are 4 parking spaces for cars, 3 of which are occupied. Then, `parkingSpaces.getEmptySpaces(CAR)` will return 1.

This exercise continues on the next page...

(d) Implement the method:

```
void ParkingArea::findVehicles(vector<string> owners)
```

This method displays the ID(s) of the parking space(s) with a vehicle whose owner is contained in the given collection `owners`.

The parking space ID must be displayed one-per-line, by following their order in `parkingSpaces`.

For example, suppose that we have a vector `v` containing the strings `"Alice"` and `"Bob"`. Then, `parkingSpaces.findVehicles(v)` will display the IDs of all parking spaces whose vehicle has an owner equal to either `"Alice"` or `"Bob"`.

Hints on using maps and vectors (See also: <https://en.cppreference.com/w/cpp/container/map> and <https://en.cppreference.com/w/cpp/container/vector>)

- To remove an element from a map or a vector, you can use their `erase(...)` methods:
 - <https://en.cppreference.com/w/cpp/container/map/erase>
 - <https://en.cppreference.com/w/cpp/container/vector/erase>

Note that, when calling `erase(...)`, references and iterators to the erased elements are invalidated. An iterator following the last erased element is returned by that method.

- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a `pair` with the map key as `p.first`, and the corresponding value as `p.second` (see <https://en.cppreference.com/w/cpp/utility/pair>).

EXERCISE 4. AUTO-SUMMING BUFFER

Daisy is writing a program that stores and retrieves `integer` values from a sensor.

Her program needs to implement an *auto-summing* buffer that accumulates values up to a maximum capacity n ; when n is exceeded, the oldest value within the buffer capacity is replaced by the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.

Therefore, she plans a `AutoSummingBuffer` class with the following interface:

- `write(v)`: appends value `v` to the buffer; if, after appending `v`, the buffer exceeds the maximum capacity, the oldest value within the buffer capacity is replaced by the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.
- `read()`: removes the oldest value from the buffer and returns it; if no value is in the buffer, it returns the default value specified in the `AutoSummingBuffer` constructor (see below).
- `occupancy()`: returns the number of elements in the buffer; calling `write()` increases the occupancy by 1 until the buffer capacity is reached, while calling `read()` decreases the occupancy by 1.
- `occurrences(v)`: returns the number of occurrences of value `v` in the buffer.

Her first test program is in file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. All files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write(v)`, `read()`, and `occupancy()`.

Example. Once completed, the class `AutoSummingBuffer` must work as follows:

- suppose that we create `buf = AutoSummingBuffer(3,-1)` (i.e. `buf` has a capacity of 3 elements and a default value `-1`);
- suppose that `buf.write(9)` is invoked, followed by `buf.write(1)`. Then, a call to `buf.occurrences(9)` must return 1, since 9 is present only once in the buffer. Also, a call to `buf.occurrences(1)` must return 1, and a call to `buf.occupancy()` must return 2;
- now, suppose that `buf.read()` is invoked. Then, a call to `buf.occurrences(9)` must now return 0, and a call to `buf.occupancy()` must return 1, since 9 was the oldest value in the buffer and it was removed when `buf.read()` was invoked.;

This exercise continues on the next page...

- now, suppose that `buf.write(3)` is invoked, followed by `buf.write(8)`, `buf.write(6)` and `buf.write(6)`. Then, a call to `buf.occupancy()` must return 3 since the buffer capacity was reached. A call to `buf.occurrences(1)` must now return 0, since 1 is beyond the buffer capacity. A call to `buf.occurrences(8)` must return 0, and a call to `buf.occurrences(12)` must return 1, since 8 has been replaced by 12 (i.e., the sum of 1, 3, and 8). A call to `buf.occurrences(6)` must return 2, since 6 is present twice in the buffer;
- now, suppose that `buf.write(5)` is invoked. Then, a call to `buf.occurrences(12)` must now return 0, and a call to `buf.occurrences(18)` must return 1, since the oldest value within the buffer capacity is no longer 12, but 18 (i.e., the sum between 1, 3, 8 and 6);
- finally, suppose that `buf.read()` is invoked. Then, a call to `buf.occurrences(18)` must now return 0, and a call to `buf.occupancy()` must return 2, since 18 was removed from the buffer when `buf.read()` was invoked.

Tasks. Help Daisy by completing the following tasks. **IMPORTANT:** for these tasks you are required to submit *both* files `ex04-library.h` and `ex04-library.cpp` enclosed in a ZIP archive.

NOTE: you are free to define the `private` members of `AutoSummingBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write(v)`, `read()`, `occupancy()` and `occurrences(v)`. When the occupancy of the buffer is at its maximum, you are also free to compute the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity whenever `write()` is invoked, or to do so when `read()` is invoked.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `AutoSummingBuffer` that extends `Buffer`. This task is completed (and passes Autolab tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `AutoSummingBuffer` that takes 2 parameters:
 - (i) an `unsigned integer` representing the maximum buffer capacity; and
 - (ii) a value of type `int` representing a default (it is used in point (c) below);
 2. in `AutoSummingBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` method to the class interface:
 - `unsigned int occurrences(int v)`
 3. finally, write a placeholder implementation of all the `AutoSummingBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).

This exercise continues on the next page...

- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void AutoSummingBuffer::write(int v)
unsigned int AutoSummingBuffer::occupancy()
```

Their intended behaviour is that each time `write()` is invoked, the value returned by `occupancy()` increases by 1, until the maximum capacity is reached (such maximum capacity is specified in the constructor — see point (a)1(i) above).

Special case: if, after invoking `write(v)`, the maximum capacity is exceeded, the oldest value within the buffer capacity must be replaced by the sum of that value and the oldest values beyond the buffer capacity.

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int AutoSummingBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1.

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)1(ii) above).

- (d) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
unsigned int AutoSummingBuffer::occurrences(int v)
```

When `occurrences(v)` is invoked, it returns the number of occurrences of value `v` in the buffer.